

# Aupulens Data Migration

## Current Implementation Status & CTO Gap Report

**Date:** October 2026 **Type:** Strict Read-Only Current-State Audit

---

### 1. Executive Summary

This audit reviews the current state of the Aupulens Data Migration implementation against the CTO Product Requirement Specification. The core pipeline (Upload -> Parse -> Map -> Validate -> Preview -> Migrate) is functionally structured and integrated into the Aupulens ERP. However, critical gaps remain before it can be considered production-ready for complex ERP migrations, notably: **Lack of Relationship Preservation**, a **UI-dependent Background Worker**, and missing **Post-Migration Verification**.

### 2. Audit Scope

The audit examined the following locations without modifying the codebase:

- `app/migration/*` (Frontend UI)
- `app/api/migration/*` (Backend API Routes)
- `lib/migration/*` (Core logic, source adapters, entity handlers, workers)
- `models/admin/Migration*` (MongoDB schemas)

### 3. Current Architecture

The architecture is a Next.js App Router full-stack implementation:

- **Frontend:** React components managing state machine (Upload -> Report).
- **Backend:** API routes coordinating parsing, mapping, and worker chunks.
- **Database:** 3 MongoDB models: `MigrationBatch`, `MigrationJob`, `MigrationRecord`.

### 4. Current Migration Flow

- **Upload:** User uploads file.
- **Parsing:** API detects type, extracts records via `sourceAdapters.ts`.
- **Mapping:** AI suggests mapping via `/mapping/route.ts`.
- **Validation:** Worker validates required fields and types, detects duplicates.
- **Preview/Resolution:** User resolves duplicates ( `PreviewAndResolution.tsx` ).
- **Migration:** Worker executes DB writes.
- **Report:** Downloadable text report.

### 5. Current UI Status

**Status:** PARTIAL The wizard UI ( `[id]/page.tsx` ) tracks progression. Duplicate resolution UI exists. However, there is no UI for inline editing of invalid records, and the Preview step does not render actual Aupulens module views.

### 6. Current Backend Status

**Status:** IMPLEMENTED The API endpoints exist for all lifecycle events (upload, mapping, worker, resolve, invalid, start, report).

## 7. Current Database Status

**Status:** IMPLEMENTED `MigrationBatch` tracks overall state, `MigrationJob` tracks file-level metadata, `MigrationRecord` isolates individual row state and prevents production pollution before confirmation.

## 8. File Format Support

**Status:** IMPLEMENTED Parsers exist in `sourceAdapters.ts` for CSV, XLS, XLSX, XML, JSON, and ZIP archives.

## 9. ERP Support

**Status:** PARTIAL Explicit adapters exist for Tally XML and Zoho Export ZIPs. However, deep semantic normalization of Tally-specific nested accounting trees into flat Aupulens structures is superficial.

## 10. AI Mapping Status

**Status:** IMPLEMENTED AI endpoint ( `/mapping` ) receives canonical schema and sample source records, generating JSON mappings successfully.

## 11. Validation Status

**Status:** PARTIAL Validates types and required fields ( `validation.ts` ). Does NOT validate target model business rules (e.g., matching tax configurations, valid status enums beyond basics).

## 12. Relationship Status

**Status:** MISSING (CRITICAL) There is no reference resolution mechanism. A Sales Invoice that references a Customer Name will not automatically look up and link the actual `_id` of that Customer in the database.

## 13. Business Entity Status

**Status:** IMPLEMENTED Entity schemas ( `entitySchemas.ts` ) and handlers ( `importer.ts` ) are defined for: Customer, Vendor, Product, Account, Employee, SalesInvoice, PurchaseInvoice, Payment, Expense.

## 14. Migration Preview Status

**Status:** PARTIAL Shows statistical summaries and duplicate resolution UI. Missing the CTO requirement of showing actual Aupulens-style read-only table views of the incoming data.

## 15. Editing / Review Status

**Status:** PARTIAL API endpoint `/invalid/route.ts` exists for inline editing, but no frontend UI component exists to interact with it.

## 16. Duplicate Handling Status

**Status:** IMPLEMENTED Worker detects duplicates against real DB ( `existingFilter` ). UI ( `PreviewAndResolution.tsx` ) allows Skip, Merge/Update, or Force Create.

## 17. Background Worker Status

**Status:** BROKEN / PARTIAL (CRITICAL) `worker.ts` processes data in chunks of 500. However, it relies on the frontend `useEffect` polling `setTimeout(triggerWorker, 100)`. If the user closes the browser, migration halts. Not a true durable queue.

## 18. Scalability Status

**Status:** NOT VERIFIED Uses iterative `.create()` and `.findByIdAndUpdate()` rather than `bulkWrite`. Has not been stress-tested with 100k+ records. Memory exhaustion is possible during Tally XML parsing.

## 19. Verification Status

**Status:** MISSING No post-migration cross-reference checks comparing source totals against target DB totals.

## 20. Report Generation Status

**Status:** PARTIAL Generates a raw `.txt` file summary ( `/report/route.ts` ). Missing PDF/CSV detailed outputs.

## 21. Security Status

**Status:** IMPLEMENTED All APIs enforce `session?.user?.tenantId` isolation. Records are correctly scoped.

## 22. UI/Design Status

**Status:** PARTIAL Uses existing Tailwind classes, but lacks standard Aupulens data-grid designs for the preview step.

## 23. Regression Status

**Status:** NOT VERIFIED No automated integration tests confirm that importing fake records does not break the live app.

## 24. Testing Evidence

**Status:** NOT VERIFIED Code exists, but there is no evidence of a standardized, automated test suite (e.g., Jest/Cypress) validating 50k+ rows or relationship preservation.

---

## 25. CTO Requirement Matrix

| Requirement                        | Current Status | Evidence / File                                                   |
|------------------------------------|----------------|-------------------------------------------------------------------|
| File Formats (CSV, XML, JSON, ZIP) | IMPLEMENTED    | <code>sourceAdapters.ts</code>                                    |
| Tally / Zoho Detection             | IMPLEMENTED    | <code>sourceAdapters.ts</code>                                    |
| Migration Workspace                | IMPLEMENTED    | <code>MigrationBatch</code> , <code>MigrationRecord</code> models |
| AI Mapping                         | IMPLEMENTED    | <code>[id]/mapping/route.ts</code>                                |

|                             |             |                                              |
|-----------------------------|-------------|----------------------------------------------|
| Deterministic Validation    | PARTIAL     | worker.ts, missing business rules            |
| Duplicate Resolution        | IMPLEMENTED | PreviewAndResolution.tsx, resolve/route.ts   |
| Pre-migration Editing       | PARTIAL     | API exists (invalid/route.ts), UI missing    |
| Relationship Preservation   | MISSING     | importer.ts lacks reference lookup logic     |
| Real Aupulens Preview       | MISSING     | UI only shows summary cards                  |
| Background Migration        | BROKEN      | Requires browser to stay open to trigger API |
| Post-Migration Verification | MISSING     | No verification logic present                |
| Downloadable Report         | PARTIAL     | .txt endpoint only                           |
| Tenant Isolation            | IMPLEMENTED | Enforced in all API routes                   |

## 26. Fully Implemented

- **File Parsing & Adapters:** Extracts structured data reliably.
- **Tenant Isolation:** Secure workspace isolation preventing cross-tenant leakage.
- **AI Schema Mapping:** Successfully maps generic columns to target models.
- **Duplicate Detection & Resolution:** UI and Backend flow for Skip/Merge/Create is complete.

## 27. Partially Implemented

- **Pre-Migration Editing:** Backend API exists, but the user cannot fix invalid rows due to missing UI.
- **Worker Execution:** Chunking logic works, but architectural trigger mechanism is fundamentally flawed (client-dependent).

## 28. Missing / Broken

- **Relationship Preservation (Missing):** Foreign keys (e.g. associating an Invoice to a Customer) are not dynamically resolved.
- **Background Worker Architecture (Broken):** Serverless architecture will timeout or halt if client disconnects.
- **Post-Migration Verification (Missing):** No integrity checking post-insertion.
- **Real Preview UI (Missing):** CTO requires actual Aupulens read-only interfaces.

## 29. Critical Risks

1. **Worker Halting:** If a user closes the tab during a 50k row migration, the process silently dies.
2. **Orphaned Records:** Lack of relationship preservation means transactions will migrate without customer/vendor context, breaking accounting ledgers.
3. **Database Thrashing:** Lack of `bulkWrite` will cause significant DB load and slow migrations for large files.

## 30. P0/P1/P2/P3 Remaining Work

- **P0 - Relationship Resolution:** Must map names/identifiers to real DB ObjectIds before insert.
- **P0 - Durable Background Worker:** Must use a true background queue (e.g., BullMQ, Inngest, Google Cloud Tasks) independent of UI polling.
- **P1 - Pre-Migration Inline Edit UI:** Build the React table for fixing invalid records.

- **P1 - BulkWrite Optimization:** Refactor `importer.ts` loop to use MongoDB `bulkWrite` .
- **P2 - Post-Migration Verification:** Implement cross-check reporting.
- **P3 - PDF/CSV Reports:** Upgrade text reports to richer formats.

## 31. Final Current-State Assessment

The Data Migration engine has successfully laid the groundwork for secure, isolated, AI-assisted imports. The data ingestion, mapping, and validation pipelines are structurally sound. However, the system is **NOT YET CTO-COMPLIANT**. It cannot safely migrate complex relational ERP datasets (like full Tally backups) due to the lack of foreign-key relationship preservation, and its execution model cannot scale to large datasets without a true background queuing architecture.